Research note

Huang Hsin

October 29, 2024

Abstract

This is a research note for implementing quasi-Newton method on minimax problem.

1 Introduction

2 Preliminary

We focus on the nonconvex-nonconcave minimax problem:

$$\min_{x\in\mathbb{R}^n}\max_{y\in\mathbb{R}^m} f(x,y) \tag{1}$$

where $f:\mathbb{R}^{n+m}\to\mathbb{R}$ is twice continuously differentiable, i.e. $f\in\mathcal{C}^2$. Here, we define the local optimal solution of this problem as follow:

Definition 1 (strict local minimax(SLmM), [**?**])**.** *(x^*,y^*) is a strict local minimax point (SLmM) of a twice differentiable function f if it is a stationary point, $\partial_{yy}f(x^*,y^*)\prec 0$ and $\mathbf{D}_{xx}f(x^*,y^*)\succ 0$*

Such definition has appeared in many research focusing on local convergence of minimax problem.

Now, we seperate the problem into two subproblem, the inner problem solving $\Phi(x)=\max_{y\in\mathbb{R}^m} f(x,y)$ and the outer problem solving $\min_{x\in\mathbb{R}^n}\Phi(x)$.

2.1 Inner problem

For the inner problem, which is a simple one-variable maximization problem $\max_{y\in\mathbb{R}^m} f(x,y)$ where the x is fixed. There are already many well-known solutions for this kind of problem. Such as gradient ascent:

$$y_{i+1}=y_i+\alpha_i\partial_y f(x,y_i) \tag{2}$$

or Newton method:

$$y_{i+1}=y_i-\alpha_i(\partial_{yy}f(x,y_i))^{-1}\partial_y f(x,y_i) \tag{3}$$

Idealy, we want to solve the inner problem accurately to give a precise function $\Phi(x)$ to the outer problem. However, in practice, it is time consuming to solve inner problem precisely. Therefore, instead of solving the inner problem exactly, we run several update to get the approximation of function Φ.

2.2 Outer problem

For the outer problem $\min_{x\in\mathbb{R}^n}\Phi(x)$, since we are not minimizing the fucntion f directly, we need to analyze the function $\Phi(x)=\max_{y\in\mathbb{R}^m} f(x,y)$ furthermore.

At a SLmM (x^*,y^*), We know that $\partial_y f(x^*,y^*(x))=0$ and $\partial_{yy}f(x^*,y^*)\prec 0$ by definition. By implict theorem, we know that there are neighborhoods $\mathcal{N}(x^*)\subset\mathbb{R}^n$, $\mathcal{N}(y^*)\subset\mathbb{R}^m$ and a continuously differentiable function

$$r:\mathcal{N}(x^*)\to\mathcal{N}(y^*)\text{ s.t. }\partial_y f(x,r(x))=0 \tag{4}$$

where $r(x)$ is a local minimizer of $f(x,\cdot)$ and $\Phi(x)=f(x,r(x))$. Therefore, on $\mathcal{N}(x^*)$:

$$\nabla\Phi(x)=\partial_x f(x,r(x))+\nabla r(x)\partial_y f(x,r(x))=\partial_x f(x,r(x)) \tag{5}$$

For the second derivative $\nabla^2\Phi(x)$. Since $\partial_y f(x, r(x)) = 0$, we know that

$$\partial_{xy} f(x, r(x)) + \nabla r(x)\partial_{yy} f(x, r(x)) = 0$$
$$\nabla y^*(x) = -\partial_{xy} f(x, y^*(x))(\partial_{yy} f(x, y^*(x)))^{-1}$$

From $\nabla\Phi(x) = \partial_x f(x, y^*(x))$, we know that

$$\begin{aligned}\nabla^2\Phi(x) &= \partial_{xx} f(x, y^*(x)) + \nabla y^*(x)\partial_{yx} f(x, y^*(x)) \\ &= \partial_{xx} f(x, y^*(x)) - \partial_{xy} f(x, y^*(x))(\partial_{yy} f(x, y^*(x)))^{-1}\partial_{yx} f(x, y^*(x))\end{aligned} \tag{6}$$

By using equation 3, the Newton update will be:

$$x^{k+1} = x^k - (\nabla^2\Phi(x^k))^{-1}\nabla\Phi(x^k) \tag{7}$$

3 Quasi-Newton method

Since calculating the Hessian used in Newton method might be time consuming and gradient descent(ascent) converge slowly, we can try to use quasi-Newton method. Quasi-Newton method is a set of methods that approximate the Hessian by using only gradient to avoid expensive computation. As we discussed in section 2.1, we can use the normal quasi-Newton method directly on the inner problem:

$$y_{i+1} = y_i + \alpha_i H_i \partial_y f(x, y_i) \tag{8}$$

where H_i is the approximation of the inverse of Hessian. Notice that we use the subscript i rather than k since if we set $H_0 = I$ and run the update only once, then it will be a simple gradient ascent step. Therefore, we will run the quasi-Newton step several time to let the approximation of Hessian become more accurate.
For outer problem,

$$x^{k+1} = x^k - B(x^k)^{-1}\nabla\Phi(x^k) = x^k - H(x^k)\nabla\Phi(x^k) \tag{9}$$

where $B(x)$ is the approximate Hessian and $H(x)$ is the inverse of approximate Hessian.

Here, we have many option for the approximation such as BFGS.
In normal BFGS for a minimization problem $\min z(x)$, we approximate the Hessian by the step difference s and the gradient difference g.

$$H_{k+1} = (I - \frac{s_k g_k^T}{g_k^T s_k})H_k(I - \frac{g_k s_k^T}{g_k^T s_k}) + \frac{s_k s_k^T}{g_k^T s_k},\ s_k = x_{k+1} - x_k,\ g_k = \nabla z_{k+1} - \nabla z_k \tag{10}$$

Come back to the minimax problem. For the inner problem, we are simply doing a maximize version quasi-Newton method since we are maximizing a function $\max_{y\in\mathbb{R}^m} f(x, y)$. For the outer problem, we are doing $\min_{x\in\mathbb{R}^n}\Phi(x)$, so we use the gradient $\nabla\Phi(x) = \partial_x f(x, y^*(x))$ in our gradient step.

4 Algorithm

Algorithm 1 is gradient-descent-BFGS(GD-BFGS) where we use gradient descent for outer problem and BFGS for inner problem.

Algorithm 1 Gradient-descent-BFGS

(step size α_x, α_y, initialization x_0, y_0)
GDBFGS$(\alpha_x, \alpha_y, x_0, y_0)$:
for $k = 0, 1, 2, 3...$ **do**
  $x_{k+1} \leftarrow \alpha_x \partial_x f(x_k, y_k)$
  $y_{k+1} \leftarrow$ **InnerBFGS**(α_y, x_{k+1}, y_k)
end for
return x_k, y_k

Algorithm 3 is complete-BFGS(CBFGS) where we use BFGS for both outer and inner problem. Notice that Algorithm 1 and 3 share the same inner loop (Algorithm 2).

Algorithm 2 InnerBFGS

InnerBFGS(α_y, x, y):
$H_0 \leftarrow I$, $y_0 \leftarrow y$
for $i = 0, 1, 2, 3...$ **do**
  $s_i \leftarrow \alpha_x H_k \partial_y f(x, y_i)$
  $y_{i+1} \leftarrow y_i + s_i$
  $g_i \leftarrow -\partial_y f(x, y_{i+1}) + \partial_y f(x, y_i)$
  $H_{i+1} \leftarrow (I - \frac{s_i g_i^T}{g_i^T s_i}) H_i (I - \frac{g_i s_i^T}{g_i^T s_i}) + \frac{s_i s_i^T}{g_i^T s_i}$
end for
return y_i

Algorithm 3 Complete-BFGS

(step size α_x, α_y, initialization x_0, y_0)
CBFGS$(\alpha_x, \alpha_y, x_0, y_0)$:
$H_0 \leftarrow I$
for $k = 0, 1, 2, 3...$ **do**
  $s_k = -\alpha_x H_k \partial_x f(x_k, y_k)$
  $x_{k+1} = x_k + s_k$
  $g_k = \partial_x f(x_{k+1}, y_k) - \partial_x f(x_k, y_k)$
  $H_{k+1} = (I - \frac{s_k g_k^T}{g_k^T s_k}) H_k (I - \frac{g_k s_k^T}{g_k^T s_k}) + \frac{s_k s_k^T}{g_k^T s_k}$
  $y_{k+1} =$ **InnerLoop**(x_{k+1}, y_k)
end for
return x_k, y_k

5 Experiment

For the experiment, we run well-conditioned guassian mean estimation, ill-conditioned guassian mean estimation and ill-conditioned gaussian covariance estimation.

algorithm	mean	mean(ill conditioned)	covariance
GDA	139.4 (0.05, 0.5, 1)	138.6 (0.05, 0.5, 1)	146.9 (0.02, 0.2, 1)
GDN	151.3 (0.05, 1.0, 1)	150.3 (0.05, 1.0, 1)	217.0 (0.02, 1.0, 1)
GDBFGS	151.6 (0.05, 1.0, 10)	147.2 (0.05, 1.0, 10)	176.5 (0.02, 1.0, 10)
CN	177.9 (1.0 , 1.0, 1)	173 (1.0 , 1.0, 1)	314 (1.0 , 1.0, 10)
CBFGS	142.9 (0.5 , 1.0, 10)	144 (0.5 , 1.0, 10)	198.0 (0.5 , 1.0, 20)
BFGSN	150.0 (0.5 , 1.0, 1)	146.4 (0.5 , 1.0, 1)	211.5 (0.7 , 1.0, 1)

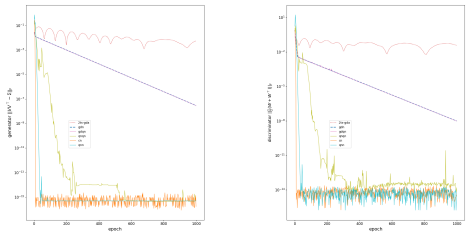
Table 1: Time spent for different model and algorithm(unit: second per 1000 epochs).
Note: the numbers brackets are $\alpha_x, \alpha_y, \#$ of inner loop

5.1 well-conditioned guassian mean

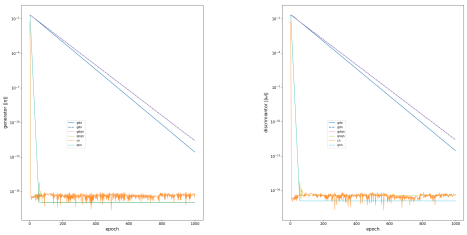
In well-conditioned guassian mean estimation, we have only 2 parameters for both generator and discriminator. Our benchmark GDA converge linearly. GDN and GDBFGS also converge linearly perform a little bit worse than GDA while spending a little bit more time. CBFGS and BFGSN converge superlinearly and CN converge even faster than CBFGS. However, CN do spend more time per epoch while CBFGS and BFGSN spends similar time to GDA, GDN and GDBFGS.

5.2 ill-conditioned guassian mean

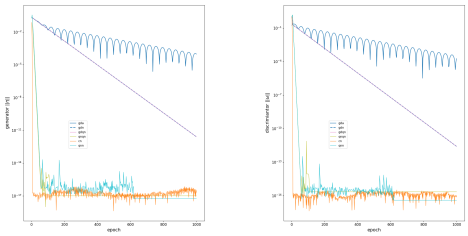
In ill-conditioned guassian mean estimation, we also have only 2 parameters for both generator and discriminator. The benchmark GDA start to perform weird. The others algorithm have similar performance in well-conditioned one. However, we can see that CN start spending more time compare to



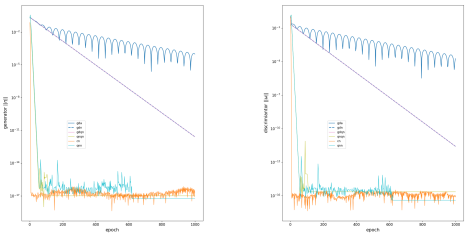
(a) Graph 1



(b) Graph 2



(c) Graph 3



(d) Graph 4

Figure 1: Multiple Graphs in One Figure

well-conditioned one. Note: When using BFGS in training generator, we can't no longer use step size $\alpha_x = 1$ in this condition anymore, otherwise, it will diverge.

5.3 covariance

In guassian covariance estimation, we have 8 parameters for both generator and discriminator. The benchmark GDA wasn't performing good. GDN and GDBFGS still perform similarly and converge linearly. However Newton method start to spend more time then quasi-Newton method. CN still converge the fastest but at the same time, it start to spend significantly more time then CBFGS (although still converge faster in time). BFGSN have a performance really close to CN and only spend a little bit more time than CBFGS. Note: Since the number of parameters increase, the time Newton method used increase significantly. However, we also need more steps to find y^* to get close to the $\Phi(x)$ when using quasi-Newton method for inner problem.

6 Future work

- try another midium scale problem (in J-symmetrix)
- try MNIST
- modify the condition number

7 Other Note

- generator is the minimizer which is the slow leader
- discriminator is the maximizer which is the faster follower
- I think the paper take a trick on gradient step, make the step size smaller(0.05 for outer, 0.5 for inner), so the performance of Newton type method become better.
- Finish LBFGS update.
- size 100: CN perform the best.

8 Outline

- Introduation
  - application of minimax problem
  - basic idea of solving minimax problem
  - mention uzawa's approach
  - talk about gradient based method and Newton method
  - talk about quasi-Newton method
- Preliminary
  - introduce minimax problem
  - introduce SLmM
  - analyze the gradient around SLmM